

CAMPUS VIRTUAL (SISTEDU)

DOCUMENTACIÓN DE DISEÑO DE SISTEMAS Y GUÍA DE DESARROLLO

DOCUMENTACIÓN DE DISEÑO DE SISTEMAS: CAMPUS VIRTUAL (SISTEDU)

IES 9-007 "Salvador Calafat"

Autores del Proyecto: Equipo de 3er Año - Tecnicatura Superior en Desarrollo de Software

1. INTRODUCCIÓN Y ARQUITECTURA GENERAL

Este documento sirve como especificación de diseño técnico y hoja de ruta para la construcción del Campus Virtual del IES 9-007. El sistema adopta principios de **Arquitectura Limpia (Clean Architecture)** y **SOLID**, garantizando que las reglas de negocio estén completamente aisladas de los detalles de implementación (como la base de datos o el framework web Django).

1.1 Diagrama de Capas

El sistema se organiza en capas concéntricas donde las dependencias apuntan únicamente hacia adentro:

- Capa de Dominio (Modelos):** Contiene las entidades puras del negocio (Carrera, Materia, Alumno, Profesor). No tienen lógica de presentación ni reglas de casos de uso complejos.
 - Capa de Aplicación (Servicios/Managers):** Es el núcleo lógico donde residen los casos de uso (Ej: Inscribir Estudiante a Cursada verificando correlativas, Firmar Acta de Examen, etc.).
 - Capa de Infraestructura y Presentación (Views, Forms, Templates, DB):** Detalles tecnológicos encargados de recibir peticiones HTTP, renderizar HTML a través de HTMX/Tailwind y persistir datos.
-

2. REPORTE DE ESTADO ACTUAL (STATUS REPORT)

Analizando el repositorio, el estado de avance es el siguiente:

- App `carreras` [EN DESARROLLO - AVANZADO]:

- Implementa separación SRP con un archivo por modelo (`carrera.py`, `plan\_estudio.py`, `cargo\_docente.py`) dentro de la carpeta `models/`.
 - La vinculación con `authentication.Profesor` se realiza mediante cadenas de caracteres (`authentication.Profesor`) para evitar dependencias circulares.
 - App `profesor` [INICIAL]:
 - Define un modelo simple `Profesor` con validación de DNI y título.
 - *Observación*: Falta desacoplar el modelo y vincularlo a un sistema de usuarios unificado.
 - App `materias` [INICIAL - REQUIERE REFACTORIZACIÓN]:
 - Tiene un único archivo `models.py` acoplado directamente al modelo de `Profesor` mediante importación física de módulos.
 - *Observación*: Debe reestructurarse bajo el patrón de un modelo por archivo e introducir los conceptos de `Correlativa` y `Comision`.
-

3. ESPECIFICACIÓN DETALLADA DE MÓDULOS Y MODELOS

Para lograr la flexibilidad que requiere un sistema estilo SIU Guaraní, la base de datos y los modelos de Django se deben estructurar de la siguiente manera:

3.1 Módulo Estructural de la Academia (`carreras` y `materias`)

A. Modelo: `Carrera` (`carreras/models/carrera.py`)

- *Responsabilidad*: Registrar la oferta académica de la institución.
- *Atributos*:
 - `nombre` (CharField, 150)
 - `codigo` (CharField, 20, unique)
 - `descripcion` (TextField, null/blank)
 - `director` (ForeignKey a `profesor.Profesor`, SET_NULL, null/blank)

B. Modelo: `PlanEstudio` (`carreras/models/plan\_estudio.py`)

- *Responsabilidad*: Administrar las currículas temporales de cada carrera.
- *Atributos*:
 - `carrera` (ForeignKey a `Carrera`)
 - `nombre` (CharField, 100) - Ej: "Plan de Estudios Resolución N° 124-2024"
 - `año\_resolucion` (PositiveIntegerField)
 - `is\_active` (BooleanField, default=True)

C. Modelo: `Materia` (`materias/models/materia.py`)

- ***Responsabilidad*:** Representar una asignatura teórica dentro del plan de estudios.
- ***Atributos*:**
 - ``nombre`` (CharField, 200)
 - ``codigo`` (CharField, 50, unique) - Ej: "DS-301"
 - ``año_cursada`` (PositiveSmallIntegerField) - Del 1 al 3 (según el año de la carrera en que se dicta)
 - ``cuatrimestre`` (CharField, choices) - Ej: ['1C', '2C', 'ANUAL']
 - ``carga_horaria`` (PositiveIntegerField) - Total de horas reloj
 - ``plan_estudio`` (ForeignKey a ``carreras.PlanEstudio``)

D. Modelo: ``MateriaCorrelativa`` (``materias/models/correlativa.py``)

- ***Responsabilidad*:** Definir las dependencias jerárquicas entre asignaturas.
 - ***Atributos*:**
 - ``materia_destino`` (ForeignKey a ``Materia``, `related_name='requisitos'`) - La materia que se desea cursar/rendir.
 - ``materia_requerida`` (ForeignKey a ``Materia``, `related_name='es_requisito_de'`) - La materia previa aprobada/regularizada.
 - ``tipo_requisito`` (CharField, choices) - Ej: [('REG', 'Regularizada para cursar'), ('APR', 'Aprobada para cursar'), ('FIN', 'Aprobada para rendir final')]
-

3.2 Módulo de Usuarios y Perfiles (``authentication``)

Para unificar accesos, se extiende el sistema de autenticación de Django. Se crea una aplicación llamada ``authentication`` con los siguientes modelos:

A. Modelo: ``Usuario`` (``authentication/models/usuario.py``)

- ***Responsabilidad*:** Credenciales e identidad global en el sistema. Hereda de ``AbstractUser`` de Django.
- ***Atributos*:**
 - ``is_estudiante`` (BooleanField, `default=False`)
 - ``is_profesor`` (BooleanField, `default=False`)
 - ``is_administrativo`` (BooleanField, `default=False`)

B. Modelo: ``Estudiante`` (``authentication/models/estudiante.py``)

- ***Responsabilidad*:** Datos específicos de alumnos e inscripciones a carreras.
- ***Atributos*:**
 - ``usuario`` (OneToOneField a ``Usuario``)
 - ``legajo`` (CharField, unique) - Generado automáticamente
 - ``dni`` (CharField, 20, unique)
 - ``telefono`` (CharField, 50)

- `foto\_perfil` (ImageField, upload_to='perfiles/estudiantes/', null/blank)
- `carreras` (ManyToManyField a `carreras.Carrera`, a través de `InscripcionCarrera`)

C. Modelo: `InscripcionCarrera` (`authentication/models/inscripcion\_carrera.py`)

- ***Responsabilidad*:** Registrar qué carreras cursa un alumno y bajo qué plan de estudios.
 - ***Atributos*:**
 - `estudiante` (ForeignKey a `Estudiante`)
 - `carrera` (ForeignKey a `carreras.Carrera`)
 - `plan\_estudio` (ForeignKey a `carreras.PlanEstudio`)
 - `fecha\_inscripcion` (DateField, auto_now_add=True)
 - `estado` (CharField, choices) - Ej: `[('ACT', 'Activo'), ('EGR', 'Egresado'), ('ABN', 'Abandonado')]
-

3.3 Módulo de Gestión y Cursado (`academico`)

A. Modelo: `Comision` (`academico/models/comision.py`)

- ***Responsabilidad*:** Representar el dictado real y anual de una materia.
- ***Atributos*:**
 - `materia` (ForeignKey a `materias.Materia`)
 - `nombre` (CharField, 50) - Ej: "División A", "Comisión Turno Tarde"
 - `ciclo\_lectivo` (PositiveIntegerField) - Ej: 2026
 - `cupos\_maximo` (PositiveIntegerField)
 - `profesores` (ManyToManyField a `profesor.Profesor`, a través de `DocenteComision`)

B. Modelo: `DocenteComision` (`academico/models/docente\_comision.py`)

- ***Responsabilidad*:** Vincular profesores a comisiones específicas y definir su cargo dentro del aula.
- ***Atributos*:**
 - `profesor` (ForeignKey a `profesor.Profesor`)
 - `comision` (ForeignKey a `Comision`)
 - `rol` (CharField, choices) - Ej: `[('TIT', 'Titular'), ('ADJ', 'Adjunto'), ('JTP', 'Jefe de Trabajos Prácticos')]

C. Modelo: `InscripcionComision` (`academico/models/inscripcion\_comision.py`)

- ***Responsabilidad*:** Registrar qué alumnos están cursando qué comisiones en el ciclo lectivo actual.
- ***Atributos*:**
 - `estudiante` (ForeignKey a `authentication.Estudiante`)
 - `comision` (ForeignKey a `Comision`)
 - `fecha\_inscripcion` (DateTimeField, auto_now_add=True)

- `estado\_cursada` (CharField, choices) - Ej: `[('CUR', 'Cursando'), ('REG', 'Regularizado'), ('APR', 'Aprobado por Promoción'), ('LIB', 'Libre')]

D. Modelo: `NotaParcial` (`academico/models/nota\_parcial.py`)

- ***Responsabilidad*:** Almacenar calificaciones de exámenes parciales e instancias de recuperación.
 - ***Atributos*:**
 - `inscripcion\_comision` (ForeignKey a `InscripcionComision`)
 - `instancia` (CharField, choices) - Ej: `[('1P', 'Primer Parcial'), ('2P', 'Segundo Parcial'), ('R1', 'Recuperatorio Primer Parcial'), ('R2', 'Recuperatorio Segundo Parcial')]
 - `calificacion` (DecimalField, max_digits=4, decimal_places=2)
-

3.4 Módulo de Mesas de Examen y Consultas (`mesas`)

A. Modelo: `MesaExamen` (`mesas/models/mesa\_examen.py`)

- ***Responsabilidad*:** Estructurar los llamados a exámenes finales (bedelía).
- ***Atributos*:**
- `materia` (ForeignKey a `materias.Materia`)
- `fecha\_hora` (DateTimeField)
- `aula` (CharField, 100)
- `llamado` (CharField, 100) - Ej: "Julio-Agosto 1° Turno"
- `presidente` (ForeignKey a `profesor.Profesor`)
- `vocal\_1` (ForeignKey a `profesor.Profesor`)
- `vocal\_2` (ForeignKey a `profesor.Profesor`)
- `estado` (CharField, choices) - Ej: `[('ABI', 'Inscripción Abierta'), ('CER', 'Inscripción Cerrada'), ('ACT', 'Acta Generada')]

B. Modelo: `InscripcionMesaFinal` (`mesas/models/inscripcion\_mesa\_final.py`)

- ***Responsabilidad*:** Controlar las solicitudes de examen final de los estudiantes.
- ***Atributos*:**
- `estudiante` (ForeignKey a `authentication.Estudiante`)
- `mesa\_examen` (ForeignKey a `MesaExamen`)
- `condicion` (CharField, choices) - Ej: `[('REG', 'Regular'), ('LIB', 'Libre')]
- `nota\_final` (DecimalField, null=True, blank=True)
- `libro` (CharField, 50, null/blank)
- `folio` (CharField, 50, null/blank)

C. Modelo: `MesaConsulta` (`mesas/models/mesa\_consulta.py`)

- ***Responsabilidad*:** Permitir que el docente publique sus horarios de clases de consulta y apoyo.
 - ***Atributos*:**
 - ``profesor`` (ForeignKey a ``profesor.Profesor``)
 - ``materia`` (ForeignKey a ``materias.Materia``)
 - ``fecha_hora`` (DateTimeField)
 - ``lugar_o_link`` (CharField, 255) - Aula física o dirección de Meet/Zoom
 - ``temario`` (CharField, 200)
 - ``alumnos_asistentes`` (ManyToManyField a ``authentication.Estudiante``, blank=True)
-

3.5 Módulo de Trámites y Certificados (``tramites``)

A. Modelo: ``SolicitudTramite`` (``tramites/models/solicitud_tramite.py``)

- ***Responsabilidad*:** Registrar peticiones administrativas de certificados (Alumno Regular, Analítico).
 - ***Atributos*:**
 - ``estudiante`` (ForeignKey a ``authentication.Estudiante``)
 - ``tipo_tramite`` (CharField, choices) - Ej: ``[(('CAR', 'Certificado Alumno Regular'), ('ANA', 'Analítico Parcial'), ('EXA', 'Certificado de Examen'))]`
 - ``fecha_solicitud`` (DateTimeField, auto_now_add=True)
 - ``estado`` (CharField, choices) - Ej: ``[(('PEN', 'Pendiente de Aprobación'), ('APR', 'Aprobado y Generado'), ('REC', 'Rechazado'))]`
 - ``documento_pdf`` (FileField, upload_to='certificados/', null/blank) - Archivo generado digitalmente por bedelía
-

4. GUÍA DE DESARROLLO PASO A PASO (FACETAS Y TAREAS)

Para organizarse de manera óptima en el desarrollo en paralelo, dividan el trabajo en las siguientes fases lógicas:

Fase 1: Core de Usuarios y Perfiles (Desarrollo: Compañero A)

- Tarea 1.1: Crear la aplicación ``authentication`` con ``django-admin startapp authentication``.
- Tarea 1.2: Implementar el modelo ``Usuario`` personalizado heredando de ``AbstractUser`` e indicar a Django que lo utilice agregando ``AUTH_USER_MODEL = 'authentication.Usuario`` en ``settings.py``.
- Tarea 1.3: Implementar los modelos de ``Estudiante`` e ``InscripcionCarrera``.
- Tarea 1.4: Escribir las migraciones de ``authentication`` y ejecutarlas.

Fase 2: Reestructuración de la Oferta Académica (Desarrollo: Compañero B)

- Tarea 2.1: Crear en `carreras` las migraciones correspondientes a los modelos ya definidos (`Carrera`, `PlanEstudio`, `CargoDocente`).
- Tarea 2.2: Reestructurar la aplicación `materias`: eliminar `models.py` y crear el subdirectorio `models/` estructurado con `\_\_init\_\_.py`, `materia.py` y `correlativa.py` (desacoplados mediante strings).
- Tarea 2.3: Registrar y ejecutar las migraciones en la base de datos local SQLite.

Fase 3: Gestión de Cursado y Comisiones (Desarrollo: Ambos en Coordinación)

- Tarea 3.1: Crear la aplicación `academico`.
- Tarea 3.2: Implementar los modelos de `Comision`, `DocenteComision`, `InscripcionComision` y `NotaParcial`.
- Tarea 3.3: Diseñar e implementar el Caso de Uso / Servicio de Inscripción (Ver sección 5 del documento).
- Tarea 3.4: Escribir test unitarios para el servicio de inscripción para asegurar que las correlativas bloqueen a los alumnos que no cumplen las materias previas.

Fase 4: Portal del Profesor y Mesas (Desarrollo: Compañero A)

- Tarea 4.1: Crear la aplicación `mesas`.
- Tarea 4.2: Definir los modelos de `MesaExamen`, `InscripcionMesaFinal` y `MesaConsulta`.
- Tarea 4.3: Diseñar la vista para que el docente pueda dar de alta sus horarios de consultas (`MesaConsulta`).

Fase 5: UI Dinámica con HTMX (Desarrollo: Compañero B)

- Tarea 5.1: En el template `base.html`, vincular la librería HTMX desde CDN.
- Tarea 5.2: Crear la vista del Plan de Estudios del alumno, inyectando el HTML con el Bento Grid del progreso académico.
- Tarea 5.3: Utilizar peticiones AJAX de HTMX (`hx-post`) para que el alumno pueda inscribirse a comisiones o mesas de consulta sin recargar la página completa.

5. CAPA DE SERVICIOS (CLEAN ARCHITECTURE)

Para no colocar la lógica de validación e inscripciones en las vistas (lo que violaría el patrón de arquitectura limpia), utilizaremos una capa de servicios.

A continuación se muestra la estructura conceptual de cómo se debe implementar el validador e inscribir a un alumno a una comisión en `academico/services/inscripcion\_service.py`:

```
# Lógica de Negocio pura en Python
class IncripcionService:
    @staticmethod
```

```
def inscribir_estudiante_a_comision(estudiante_id, comision_id):
# 1. Obtener estudiante y comision de la base de datos
# 2. Validar que la comisión tenga cupo disponible. Si no: lanzar CupoLlenoException()
# 3. Obtener las materias correlativas requeridas para cursar esta asignatura
# 4. Validar contra el HistorialAcademico del alumno que estén regularizadas/aprobadas. Si
no: lanzar CorrelativaNoCumplidaException()
# 5. Registrar al estudiante en la comisión con estado 'CUR' (Cursando)
# 6. Restar un cupo en la comisión o guardar el registro.
pass
```

Este servicio es invocado desde la vista de Django, la cual solo se encarga de manejar la petición HTTP y devolver la respuesta visual apropiada (HTML a través de HTMX o un JSON de error).

6. INTERFAZ GRÁFICA Y PRESENTACIÓN (HTMX + TAILWIND)

La página de **Plan de Carrera** del estudiante se beneficiará de la interactividad inmediata que ofrece HTMX:

1. **Estructura Bento Grid:** Muestra dinámicamente las tarjetas de Promedio, Materias Aprobadas y Porcentaje de Avance General mediante cálculos en la vista que consultan el modelo `HistorialAcademico`.
2. **Tabla de Materias por Año:** Las asignaturas deben agruparse por `año\_cursada`.
3. **Acciones Instantáneas con HTMX:**
 - Al hacer clic en una materia pendiente que tenga las inscripciones habilitadas, el botón disparará un atributo `hx-post="{% url 'inscribir\_materia' materia.id %}"` que reemplazará el estado de "Pendiente" a "En Curso" (con un spinner de carga) sin parpadear la pantalla.